

---

# LaTER: Efficient Test-Time Reasoning via Latent Exploration and Explicit Verification

---

Xuan Li<sup>1</sup> Yining Wang<sup>2</sup> Yuchen Liu Guanjun Liu<sup>1</sup> Delai Qiu<sup>2</sup>  
Shengping Liu<sup>2</sup> Jiaen Liang<sup>2</sup> Wei Huang<sup>2</sup> Jun Yu<sup>1†</sup> Junnan Zhu<sup>3†</sup>

<sup>1</sup>University of Science and Technology of China,

<sup>2</sup>Unisound AI Technology Co., Ltd,

<sup>3</sup>MAIS, Institute of Automation, Chinese Academy of Sciences  
harryjun@ustc.edu.cn, junnan.zhu@nlpr.ia.ac.cn

## Abstract

Chain-of-thought (CoT) reasoning improves large language models (LLMs) on difficult tasks, but it also makes inference expensive because every intermediate step must be generated as a discrete token. Latent reasoning reduces visible token generation by propagating continuous states, yet replacing explicit derivations with latent computation can hurt tasks that require symbolic checking. We propose **Latent-Then-Explicit Reasoning (LaTER)**, a two-stage paradigm that first performs bounded exploration in a continuous latent space and then switches to explicit CoT for verification and answer generation. In a training-free instantiation, LaTER projects final-layer hidden states back to the input embedding space, preserves the latent KV cache, and uses entropy and model-native stop-token probes to decide when to switch. We find that strong reasoning models already exhibit structured latent trajectories under this interface. On Qwen3-14B, training-free LaTER reduces total token usage by 16%–32% on several benchmarks while matching or improving accuracy on most of them; for example, it improves AIME 2025 from 70.0% to 73.3% while reducing tokens from 15,730 to 10,661. We further construct LATENT-SWITCH-69K, a supervised corpus that pairs condensed solution intuitions with shortened explicit derivations. Fine-tuning with latent rollout and halting supervision yields additional gains: trained LaTER reaches 80.0% accuracy on AIME 2025, 10.0 points above the standard CoT baseline, while using 33% fewer tokens. Our code, data, and model are available at <https://github.com/TioeAre/LaTER>.

## 1 Introduction

CoT prompting is a simple and effective way to improve reasoning in LLMs [1]. By generating intermediate derivations before the final answer, CoT improves performance on mathematics, science, and code tasks [2]. Its main drawback is cost. Strong reasoning models often produce long visible traces, and each additional token increases latency, memory traffic, and attention computation [3]. The cost is especially high when the model spends many tokens on tentative exploration, syntactic scaffolding, or discarded solution paths before reaching a stable derivation.

Recent work therefore studies reasoning in a continuous latent space [4, 5]. Instead of sampling a visible token at every reasoning step, a model can feed back a hidden state or a soft embedding as the next input, using either an analytic mapping such as a pseudo-inverse projection [6] or a learned projector [7], and only decodes discrete readable tokens in the final answer stage. This can substantially reduce visible token generation and has shown promising efficiency gains [4, 8,

---

<sup>†</sup>Corresponding author.

9]. However, pure latent reasoning also has a clear weakness: when a problem requires careful symbolic manipulation, explicit checking, or exact answer formatting, fully replacing CoT with latent computation can reduce accuracy on difficult benchmarks such as MATH-500 and AIME [10, 11, 12].

This suggests that latent and discrete reasoning should not be viewed as mutually exclusive alternatives. A more natural division of labor is to use continuous computation for early exploration and reserve discrete tokens for verification. Human solvers often behave in a similar way: they may first search mentally for a plan and only later write a step-by-step solution. We use this analogy only as motivation for a computational design. The central question is whether an LLM can spend part of its test-time computation in a high-bandwidth latent state, then return to explicit CoT when precise symbolic reasoning is most valuable.

We propose **LaTER (Latent-Then-Explicit Reasoning)**, a hybrid reasoning paradigm that separates exploration from verification. Given a prompt, LaTER first performs a bounded latent rollout. At each latent step, the final-layer hidden state is mapped back into the input embedding space and reused as the next input, without committing to a visible token. The model then switches to ordinary token generation while preserving the latent-phase KV cache, so the explicit derivation is conditioned on the preceding latent trajectory rather than starting from scratch.

We study LaTER in two settings. First, we show that no additional training is required for the interface to be useful. A training-free version uses a simple adaptive switch based on latent entropy and decoded stop-token probes. On Qwen3-14B, this already improves AIME 2025 from 70.0% to 73.3% while reducing average token usage from 15,730 to 10,661, and improves MATH-500 from 93.4% to 97.2% with 17% fewer tokens. Second, we train a LaTER model on LATENT-SWITCH-69K, a dataset designed to teach the model how to allocate latent exploration before explicit reasoning. The trained model reaches 80.0% on AIME 2025, a 10.0-point gain over the standard CoT baseline, while using 33% fewer tokens.

Our contributions are threefold. (i) We introduce a latent-then-explicit reasoning interface that preserves the latent KV cache and turns latent computation into a precursor to explicit verification. (ii) We identify training-free latent switching signals, including terminating-token probes and entropy dynamics, showing that pretrained reasoning models can already support structured latent rollouts. (iii) We construct LATENT-SWITCH-69K and train a LaTER model that improves the accuracy–efficiency tradeoff across mathematics, coding, and knowledge-intensive reasoning benchmarks.

## 2 Training-Free LaTER

We first ask whether a pretrained reasoning model can benefit from a latent-first, explicit-second procedure without any task-specific training. This setting isolates the inference-time interface from supervised adaptation. We show that strong reasoning models can perform several continuous latent steps, retain those steps in the KV cache, and then convert the accumulated state into explicit CoT with lower token usage. We also show that fixed latent horizons are brittle, motivating adaptive switching based on the model’s own latent dynamics.

### 2.1 Preliminaries and notation.

Let  $Q = (Q_1, \dots, Q_m)$  denote the prompt. At latent step  $s$ , the model produces a final-layer hidden state  $h_s \in \mathbb{R}^{d_h}$ . Instead of decoding  $h_s$  to a token ID and feeding that token back into the model, we map  $h_s$  directly into the input embedding space. Following the latent-transition construction of LatentMAS [6], we use

$$e_{s+1}^{\text{lat}} = W_a h_s, \quad W_a \approx W_{out}^\dagger W_{in}, \quad (1)$$

where  $W_{in}$  is the input embedding matrix,  $W_{out}$  is the output projection matrix, and  $W_{out}^\dagger$  denotes the pseudo-inverse of  $W_{out}$ . The vector  $e_{s+1}^{\text{lat}}$  is then used as the next-step input embedding. This produces a continuous trajectory

$$h_1 \rightarrow e_2^{\text{lat}} \rightarrow h_2 \rightarrow e_3^{\text{lat}} \rightarrow \dots \rightarrow h_S, \quad (2)$$

with no discrete token commitment at the intermediate latent positions. For diagnostics only, we decode each latent hidden state into a probe distribution and an argmax probe token,

$$p_s = \text{softmax}(W_{out} h_s), \quad \hat{y}_s = \arg \max_i (p_s(i)), \quad (3)$$

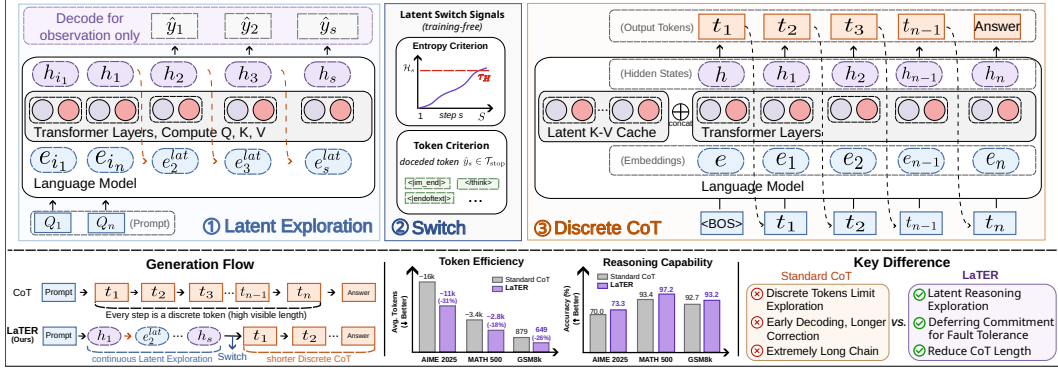


Figure 1: **Overview of training-free LaTER.** Given a user prompt, the model first enters a latent reasoning phase, where the final-layer hidden state is projected back into the input embedding space and reused as the next-step input, without committing to visible tokens. The model then switches to explicit CoT decoding, reusing the latent KV cache to generate reasoning steps and the final answer. The probe token  $\hat{y}_s$  is never used as the next input. It is only an observation of how the latent state aligns with the model’s vocabulary space. We also compute the entropy of the probe distribution,

$$\mathcal{H}_s = - \sum_i p_s(i) \log p_s(i). \quad (4)$$

which provides a scalar summary of the model’s uncertainty at that latent step.

After the latent rollout, LaTER switches to ordinary explicit CoT decoding. The switch is not a reset: we pass the latent-phase past\_key\_values into the explicit phase, so the generated derivation conditions on the latent trajectory. We evaluate two switching policies:

- **Fixed-step switching.** The model performs  $N$  latent steps and then enters explicit CoT decoding.
- **Adaptive switching.** The model exits latent reasoning when either the entropy crosses a threshold, the decoded probe token belongs to a model-specific set of terminating tokens such as `<|im_end|>`, `</think>`, or `<|endoftext|>`.

Formally, the adaptive switch is

$$\text{switch}(s) = \mathbf{1}[\mathcal{H}_s > \tau_{\mathcal{H}} \vee \hat{y}_s \in \mathcal{T}_{\text{stop}}], \quad (5)$$

where  $\tau_{\mathcal{H}}$  is an entropy threshold and  $\mathcal{T}_{\text{stop}}$  is the terminating-token set. The next subsection explains why these two signals are empirically meaningful.

## 2.2 Empirical motivation: latent trajectories are structured

A concern with latent reasoning is that hidden states might drift away from the vocabulary manifold, making repeated latent transitions unstable or semantically meaningless. Our experiments suggest a more structured picture for reasoning models such as Qwen3-14B [13], DeepSeek-R1-Distill-Llama-8B [2], and OLMo3-32B-Think [14].

**Phenomenon 1: probe tokens reveal autoregressive stopping structure.** Early latent states often decode to low-content probes, such as empty strings or repeated newline symbols (“\n\n”). After additional latent steps, however, the argmax probe frequently reaches model-native terminating symbols such as `<|im_end|>`, `</think>`, or `<|endoftext|>`. These probe tokens are not fed back into the model, so they do not drive the rollout. Their appearance instead indicates that the continuous trajectory remains coupled to the model’s generative prior. In this sense, latent reasoning does not behave like arbitrary numerical drift; it often approaches states that the language model itself would interpret as closure.

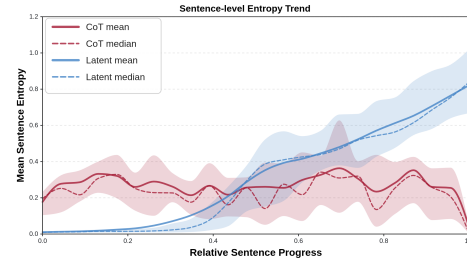


Figure 2: Entropy over normalized reasoning progress on AIME 2025 for Qwen3-14B. Blue: mean latent-reasoning entropy after aligning each example from latent start to end. Red: mean CoT entropy after normalizing each sentence by within-sentence progress.

This observation is central to LaTER. If the model internally approaches a state that resembles “ready to stop”, then switching to explicit CoT can be aligned with the model’s own trajectory and the reasoning patterns it acquired during pretraining, rather than imposed at an unrelated time.

**Phenomenon 2: entropy supports an explore-then-verify interpretation.** As shown in Figure 2, the average entropy during latent rollout tends to rise over normalized latent progress before termination. This differs from ordinary explicit decoding, where entropy is often locally high near the beginning of a sentence and then declines as syntax and previously generated words constrain the continuation. The latent phase therefore appears to support a broader and less locally constrained search, while the later explicit phase converts the accumulated state into a step-by-step derivation.

We do not claim that entropy alone fully explains latent reasoning. Rather, these two observations provide practical switching signals: the terminating-token probe suggests that the trajectory is approaching closure, and the entropy profile indicates when the latent state is entering a high-uncertainty regime. Together they motivate the adaptive rule in Eq. 5.

### 2.3 Training-free experimental setup

We compare standard discrete CoT decoding with training-free LaTER under the same prompts and decoding settings. We report accuracy and total token usage. For LaTER, token usage counts both latent steps and emitted explicit tokens, so reductions are not an artifact of ignoring latent computation. We evaluate Qwen3-14B, DeepSeek-R1-Distill-Llama-8B, and OLMo3-32B-Think on AIME 2025 [15], MATH-500 [16], GSM8K [17], GPQA [18], ARC-Challenge [19], HumanEval+, and MBPP+ [20, 21].

### 2.4 Fixed-steps switching results

For Qwen3-14B, we follow the official decoding recommendations: temperature = 0.6, top- $p$  = 0.95, top- $k$  = 20, and max\_new\_tokens = 38192. Under this setup, the standard discrete CoT baseline reaches 70.0% accuracy on AIME 2025 with roughly 16K tokens on average. Figure 3 shows that fixed-step LaTER can reduce token usage substantially, but it does not fully match the baseline accuracy. The best fixed horizons, around 50–60 latent steps, reach 63.3% accuracy with about 10K–12K total tokens.

The fixed-step curve is non-monotonic: performance first improves as the latent budget increases, then degrades when the return to explicit reasoning is delayed too long. This pattern supports the role separation behind LaTER. Latent exploration is useful up to a point, but difficult problems still benefit from an explicit symbolic phase that checks intermediate conclusions and formats the final answer. A single fixed horizon cannot adapt to instance difficulty, which motivates the adaptive switch.

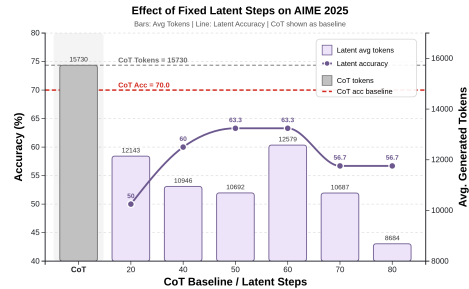


Figure 3: Accuracy and token usage on AIME 2025 as the fixed latent-step budget varies.

### 2.5 Adaptive switching results

Adaptive LaTER uses the same decoding configuration as above but replaces the fixed latent horizon with Eq. 5. At each latent step, we monitor the entropy of the probe distribution and the argmax probe token. The model switches to explicit CoT once the entropy exceeds 7 or the probe token becomes a terminating symbol such as `<|im_end|>`. Table 1 shows that this simple policy usually reduces token usage and often preserves or improves accuracy relative to the paired CoT baseline. These gains are particularly evident in stronger models and on tasks requiring extended reasoning, achieve greater token savings without compromising solution quality. The effect is strongest for Qwen3-14B: adaptive LaTER improves AIME 2025 from 70.0% to 73.3% while reducing tokens from 15,730 to 10,661, and it improves MATH-500 from 93.4% to 97.2% while reducing tokens by 17%.

Figure 5 gives a qualitative view on one AIME 2025 example. In the dominant PC1–PC2 plane, the discrete CoT trajectory is relatively scattered, suggesting that the model is still searching for a solution path. LaTER first follows a compact latent trajectory. After switching, its explicit trajectory forms repeated refinements along a shared direction rather than spreading randomly. This visualization is

Table 1: Training-free LaTER with adaptive switching across seven benchmarks, compared with the corresponding discrete CoT baseline for each backbone. Token counts include latent steps plus emitted explicit tokens. Green cells mark accuracy gains over the paired baseline, blue cells mark token reductions, and red cells mark token increases.

| Model               | Method   | AIME 2025              |                                    | GSM8K                  |                                  | MATH-500               |                                   | ARC-Challenge          |                                  | GPQA                   |                                   | HumanEval+ |                                   | MBPP+                  |                                   |
|---------------------|----------|------------------------|------------------------------------|------------------------|----------------------------------|------------------------|-----------------------------------|------------------------|----------------------------------|------------------------|-----------------------------------|------------|-----------------------------------|------------------------|-----------------------------------|
|                     |          | Acc.                   | Token                              | Acc.                   | Token                            | Acc.                   | Token                             | Acc.                   | Token                            | Acc.                   | Token                             | Acc.       | Token                             | Acc.                   | Token                             |
| DS-Distill-Llama-8B | Baseline | 33.3                   | 18546                              | 76.7                   | 498                              | 82.6                   | 2932                              | 87.7                   | 844                              | 39.9                   | 6186                              | 76.2       | 3890                              | 67.1                   | 3154                              |
|                     | LaTER    | 30.0                   | <b>11622</b> ( $\downarrow 37\%$ ) | <b>78.6</b> ( $+1.9$ ) | <b>378</b> ( $\downarrow 24\%$ ) | 79.8                   | 2927                              | 85.6                   | <b>983</b> ( $\uparrow 17\%$ )   | <b>41.9</b> ( $+2.0$ ) | <b>5896</b> ( $\downarrow 5\%$ )  | 73.1       | <b>3051</b> ( $\downarrow 22\%$ ) | <b>69.3</b> ( $+2.2$ ) | <b>2351</b> ( $\downarrow 25\%$ ) |
| Qwen3-14B           | Baseline | 70.0                   | 15730                              | 92.7                   | 879                              | 93.4                   | 3472                              | 96.3                   | 631                              | 62.6                   | 6301                              | 90.8       | 2427                              | 82.0                   | 2144                              |
|                     | LaTER    | <b>73.3</b> ( $+3.3$ ) | <b>10661</b> ( $\downarrow 32\%$ ) | <b>93.2</b> ( $+0.5$ ) | <b>649</b> ( $\downarrow 26\%$ ) | <b>97.2</b> ( $+3.8$ ) | <b>2887</b> ( $\downarrow 17\%$ ) | <b>96.5</b> ( $+0.2$ ) | <b>533</b> ( $\downarrow 16\%$ ) | 60.6                   | <b>4445</b> ( $\downarrow 29\%$ ) | 90.8       | <b>1790</b> ( $\downarrow 26\%$ ) | 79.6                   | <b>1760</b> ( $\downarrow 18\%$ ) |
| OLMo3-32B-Think     | Baseline | 60.0                   | 17042                              | 94.3                   | 971                              | 97.6                   | 4197                              | 95.0                   | 791                              | 61.6                   | 10460                             | 89.0       | 3260                              | 82.5                   | 2760                              |
|                     | LaTER    | 60.0                   | <b>13251</b> ( $\downarrow 22\%$ ) | <b>96.5</b> ( $+2.2$ ) | 882                              | 93.6                   | <b>3427</b> ( $\downarrow 18\%$ ) | <b>95.3</b> ( $+0.3$ ) | <b>882</b> ( $\uparrow 12\%$ )   | 60.1                   | <b>7145</b> ( $\downarrow 32\%$ ) | 89.0       | 3201                              | 80.9                   | 2710                              |

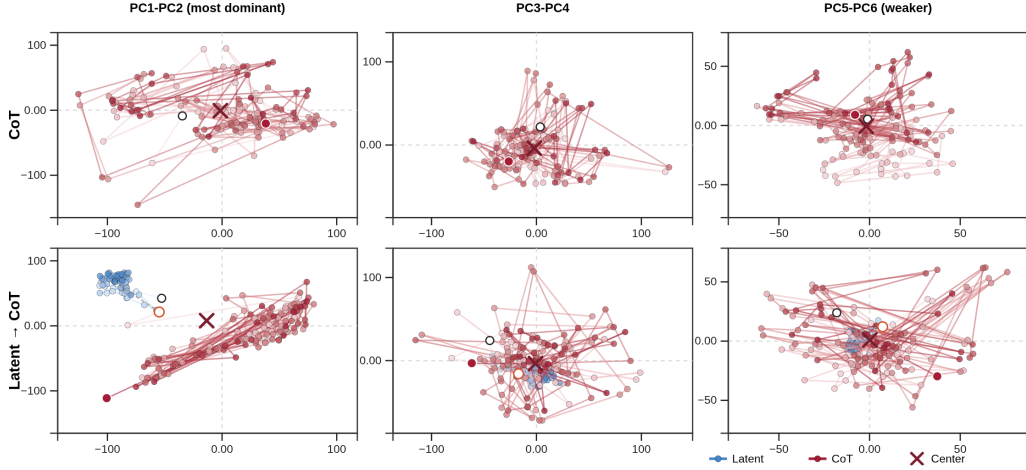


Figure 5: PCA trajectories of Qwen3-14B on an AIME 2025 example under adaptive switching. The plotted coordinates are the first six PCA components of the final-layer hidden states. Blue denotes the latent trajectory, and red denotes the first 256 steps of the explicit CoT trajectory. Color intensity increases from light to dark as reasoning progresses.

not a proof of mechanism, but it is consistent with the hypothesis that latent exploration organizes the state before explicit verification.

**Why does adaptive switching help?** A fixed horizon gives every instance the same latent budget, regardless of difficulty or internal confidence. Figures 3 and Figures 4 together show that neither too few nor too many latent steps are desirable. If the latent phase is too short, the model leaves latent exploration before the hidden state is sufficiently organized, so the later explicit CoT cannot fully benefit. If the latent phase is too long, latent computation begins to replace useful explicit verification, which hurts accuracy and can also weaken the overall efficiency–accuracy trade-off. The key is therefore to find a balanced exit point where latent exploration is sufficient but not excessive.

Adaptive switching aims to approximate this balance by using the model’s own latent dynamics—including probe entropy and terminating-token probes—to decide when to exit according to the difficulty of the current problem and the model’s internal confidence.

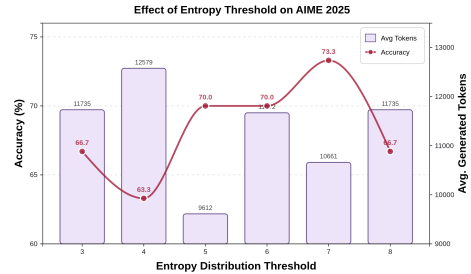


Figure 4: Effect of the entropy threshold  $\tau_H$  on training-free adaptive LaTER for Qwen3-14B on AIME 2025

## 2.6 Failure cases and the limit of hand-crafted switching

The training-free results also reveal an important limitation. On some AIME 2025 problems, latent entropy remains low and the discrete CoT baseline already solves the problem correctly; adding latent steps can still hurt. Low entropy therefore does not by itself imply that the model should remain in, or exit from, the latent phase. Appendix A then analyzes sample-level entropy trajectories. A key finding is that the model does not use the same latent entropy scale for every problem. Instead, it appears to selectively amplify the entropy of the latent phase depending on problem difficulty and

internal confidence. By contrast, in ordinary CoT decoding, the peak entropy and overall entropy range are much more similar across samples. In the latent phase, however, different samples can show more than an order-of-magnitude difference in entropy scale.

This failure mode has two implications. First, a single global entropy threshold is too coarse: different problems can require different latent horizons even when their entropy values appear similar. Second, the model should not only be monitored during latent reasoning; it should learn when to stop. The training-free setting establishes that useful switching signals exist, but it also shows that hand-crafted rules only approximate the ideal decision boundary. This motivates the supervised LaTER training procedure in the next section.

### 3 Training LaTER

The training-free study shows that pretrained models can use latent rollouts, but it also shows that hand-crafted switching is limited. We therefore train a Qwen3-14B model to use a latent segment before explicit reasoning. The trained system differs from the training-free version in two ways: it replaces the pseudo-inverse mapping with a learned projector, and it receives supervision that teaches the model how long the latent segment should be.

#### 3.1 Model architecture

We extend the tokenizer with two boundary tokens, `<latent_think>` and `</latent_think>`. The embedding layer, transformer backbone, and language-modeling head keep their original architecture, and we add a lightweight projector  $g_\phi$  that maps decoder hidden states back into the token embedding space. During latent reasoning, the model computes

$$h_t = f_\theta(e_t, \mathcal{C}_{<t}), \quad e_{t+1} = g_\phi(h_t), \quad (6)$$

where  $f_\theta$  is the transformer,  $e_t$  is the current latent input embedding, and  $\mathcal{C}_{<t}$  is the causal context. This recurrence updates the model’s internal reasoning state without emitting a visible token at each latent position. The supervised assistant format is:

$$\text{<latent\_think>} l_1, \dots, l_m \text{</latent\_think>} \text{<think>} t_1, \dots, t_n \text{</think>} a, \quad (7)$$

where  $l_i$  are latent placeholder positions,  $t_i$  are distilled explicit CoT tokens, and  $a$  is the final answer. The latent placeholders are not supervised with ordinary token-level cross-entropy (CE). Instead, their input embeddings are replaced by recurrent projector outputs, so gradients from the later explicit reasoning and answer tokens teach the model how to use them as hidden computation steps.

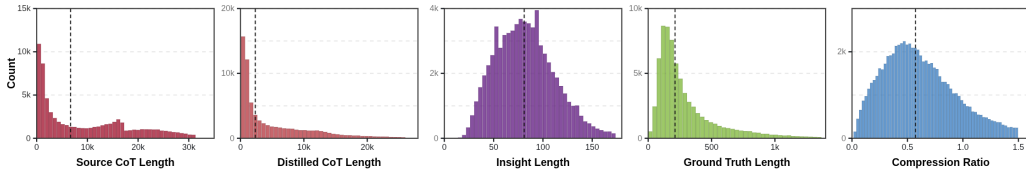


Figure 6: Token statistics of the distilled corpus. The figure compares original and distilled reasoning lengths and shows the resulting CoT compression ratios.

#### 3.2 Training data construction

We construct the supervised corpus from reasoning traces sampled from Dolci-Think-SFT-32B [14] and distill them with a stronger reasoning teacher. For each problem, the teacher produces a short *solution intuition*: a few sentences describing the high-level plan without a full derivation. The teacher then generates a shorter explicit CoT conditioned on the original problem and the solution intuition. Each retained record contains a problem, an intuition, a compressed CoT, and a final answer.

The latent budget is tied to the intuition length. If the retained intuition contains  $L$  tokens, pre-processing assigns approximately  $L/2$  latent steps, subject to maximum-length and tokenization constraints. This design uses the intuition length as a proxy for how much condensed reasoning should be represented by the latent segment.





Table 3: Comparison on the benchmarks between different baselines. Green cells mark the best accuracy among all methods for each benchmark, while blue cells mark the lowest token usage.

| Method                          | AIME 2025           |                                  | GSM8K              |                                | MATH-500           |                                 | ARC-Challenge      |                                | GPQA               |                                 | HumanEval+         |                                 | MBPP+              |                                 |
|---------------------------------|---------------------|----------------------------------|--------------------|--------------------------------|--------------------|---------------------------------|--------------------|--------------------------------|--------------------|---------------------------------|--------------------|---------------------------------|--------------------|---------------------------------|
|                                 | Acc.                | Token                            | Acc.               | Token                          | Acc.               | Token                           | Acc.               | Token                          | Acc.               | Token                           | Acc.               | Token                           | Acc.               | Token                           |
| CoT Baseline                    | 70.0                | 15730                            | 92.7               | 879                            | 93.4               | 3472                            | 96.3               | 631                            | 62.6               | 6301                            | 90.8               | 2427                            | 82.0               | 2144                            |
| CoT-SFT                         | 73.3                | 12687                            | 94.1               | 743                            | 94.0               | 3014                            | 95.5               | 578                            | 61.1               | 5885                            | 90.2               | 2071                            | 78.0               | 1817                            |
| <b>LaTER</b><br>(training-free) | 73.3 (+3.3)         | 10661 ( $\downarrow$ 32%)        | 93.2 (+0.5)        | 649 ( $\downarrow$ 26%)        | <b>97.2</b> (+3.8) | 2887 ( $\downarrow$ 17%)        | 96.5 (+0.2)        | 533 ( $\downarrow$ 16%)        | 60.6               | 4445 ( $\downarrow$ 29%)        | 90.8               | 1790 ( $\downarrow$ 26%)        | 79.6               | 1760 ( $\downarrow$ 18%)        |
| <b>LaTER</b><br>(training)      | <b>80.0</b> (+10.0) | <b>10575</b> ( $\downarrow$ 33%) | <b>94.8</b> (+2.1) | <b>644</b> ( $\downarrow$ 27%) | 96.4 (+3.0)        | <b>2541</b> ( $\downarrow$ 27%) | <b>96.7</b> (+0.4) | <b>434</b> ( $\downarrow$ 31%) | <b>63.1</b> (+0.5) | <b>3885</b> ( $\downarrow$ 38%) | <b>92.2</b> (+1.4) | <b>1776</b> ( $\downarrow$ 27%) | <b>82.8</b> (+0.8) | <b>1717</b> ( $\downarrow$ 20%) |

Only the interior reasoning tokens belong to  $\mathcal{S}_{\text{CoT}}$ . The boundary tags, latent boundary tokens, answer tokens, and `<|im_end|>` belong to  $\mathcal{S}_{\text{nonCoT}}$ . We set  $\lambda_{\text{CoT}} = 0.5$  so that explicit reasoning supervision remains useful without overwhelming the structural and answer tokens.

For self-distillation, we precompute top- $k$  teacher distributions with  $k = 128$ . The teacher is the same Qwen3-14B model used to initialize training, but queried with a different input format: we concatenate the original question prompt and the distilled solution intuition into the user message, then feed the distilled short CoT as the assistant continuation and record the teacher distribution at each token position in that short CoT. On these valid teacher positions, the student minimizes a temperature-scaled KL divergence, with temperature  $T = 1.0$  and weight  $\lambda_{\text{KL}} = 0.25$ .

$$\mathcal{L}_{\text{KL}} = \frac{1}{|\mathcal{S}_{\text{KL}}|} \sum_{i \in \mathcal{S}_{\text{KL}}} D_{\text{KL}}\left(q_i^{(T)} \parallel p_{\theta}^{(T)}(\cdot \mid x_{< i})\right) \quad (9)$$

Finally, we train the model to terminate latent reasoning at the intended boundary. Let  $\mathcal{S}_{\text{lat}}^{\text{int}}$  denote latent interior positions,  $\mathcal{S}_{\text{lat}}$  denote all latent positions,  $\mathcal{V}_{\text{forbid}}$  be the set of forbidden structural tokens, and  $b_i \in \{0, 1\}$  indicate whether  $i$  is the correct stopping boundary. The raw halt loss is

$$\mathcal{L}_{\text{halt}} = \frac{1}{|\mathcal{S}_{\text{lat}}^{\text{int}}|} \sum_{i \in \mathcal{S}_{\text{lat}}^{\text{int}}} \sum_{v \in \mathcal{V}_{\text{forbid}}} [z_{i,v} - z_{i,\text{max}}]_+ + \frac{1}{|\mathcal{S}_{\text{lat}}|} \sum_{i \in \mathcal{S}_{\text{lat}}} \text{BCE}(\sigma(z_{i, \text{<latent\_think>}}), b_i), \quad (10)$$

where  $z_{i,v}$  is the logit of token  $v$  at position  $i$ ,  $z_{i,\text{max}}$  is the largest logit among allowed non-structural tokens, and  $\sigma(\cdot)$  is the sigmoid function. The first term penalizes forbidden structural tokens when they become too competitive before the stopping point, while the second term directly trains the model to emit `</latent_think>` exactly at the correct boundary. The raw halt loss is assigned a base weight  $\lambda_{\text{halt}}^{(s)} = 0.025$ . To reduce interference between stopping supervision and the main language-modeling objective, we further modulate this term with a dynamic gate,

$$\alpha_t = \text{clip}\left(\frac{\text{EMA}(\mathcal{L}_{\text{CE}})_t}{\mathcal{L}_{\text{CE},t} + \epsilon}, 0, 1\right), \quad \mathcal{L}_{\text{halt}}^{\text{eff}} = \alpha_t \lambda_{\text{halt}}^{(s)} \mathcal{L}_{\text{halt}} \quad (11)$$

where  $\text{EMA}(\mathcal{L}_{\text{CE}})_t$  denotes the exponential moving average of the CE loss up to optimization step  $t$ ,  $\mathcal{L}_{\text{CE},t}$  is the current-step CE loss,  $\epsilon$  is a small constant for numerical stability, and  $\text{clip}(\cdot, 0, 1)$  truncates the gate to the interval  $[0, 1]$ . The effective halt loss  $\mathcal{L}_{\text{halt}}^{\text{eff}}$  therefore applies stopping supervision mainly when it does not conflict with learning the token-level prediction objective. The total training loss is

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}} + \mathcal{L}_{\text{halt}}^{\text{eff}} \quad (12)$$

### 3.4 Training setup

We use AdamW [23] with learning rate  $1.0 \times 10^{-7}$ , minimum cosine learning rate  $1.0 \times 10^{-8}$ , weight decay 0.01, and  $\beta_1 = 0.9$ ,  $\beta_2 = 0.95$ . We enable FlashAttention 2 [24]. Distributed training uses DeepSpeed ZeRO-3 [25], and the launcher uses 8\*Nvidia A800 80G GPUs on a single node.

For evaluation, we compare four systems on the same benchmark: the original Qwen3-14B with standard explicit CoT prompting, a CoT SFT model trained on the same distilled data using only the CE and KL objectives, LaTER in the training-free setting, and the fully trained LaTER model. The CoT SFT baseline uses exactly the same training data and optimization strategy as trained LaTER, but removes the latent-reasoning component and therefore does not include the  $\mathcal{L}_{\text{halt}}^{\text{eff}}$ .



### 3.5 Main results

Table 3 compares the four Qwen3-14B variants. Trained LaTER achieves the lowest token usage on all seven benchmarks and the best accuracy on most of them. On AIME 2025, it reaches 80.0% accuracy, 10.0 points above the standard CoT baseline, while reducing average token usage by 33%. It also improves GSM8K, ARC-Challenge, GPQA, HumanEval+, and MBPP+ relative to the baseline while using fewer tokens.

**Transcending the Accuracy-Efficiency Trade-Off.** A key comparison is CoT-SFT versus trained LaTER. CoT-SFT benefits from the same distilled data and improves AIME 2025 from 70.0% to 73.3%, but it remains less accurate than trained LaTER and uses more tokens (12,687 versus 10,575 on AIME 2025). This suggests that the gains are not merely a consequence of shorter supervised traces: the latent-first architecture contributes additional efficiency and reasoning accuracy.

**Isolating the Role of Latent Reasoning.** The results are also nuanced. Training-free LaTER remains the strongest method on MATH-500 in Table 3, whereas trained LaTER is more efficient and stronger on most other tasks. This indicates that supervised latent training improves the overall accuracy–efficiency frontier but does not uniformly dominate every benchmark. We view this as evidence that latent-budget allocation and data mixture remain important design choices.

## 4 Related Works

**Training-free latent reasoning.** Soft Thinking and SwiReasoning [26, 27] replace hard token inputs with probability-weighted mixtures of token embeddings, enabling latent reasoning from the model’s own next-token distribution. However, soft-embedding methods can collapse toward the dominant token and thus behave similarly to greedy decoding, limiting their ability to maintain alternative reasoning paths. SeLaR [28] addresses this issue with entropy-gated activation, applying latent reasoning only at high-uncertainty steps and preserving discrete decoding at deterministic steps. LatentMAS does not use next token embedding mixtures [6]. Instead, it projects the previous step’s hidden state back into the input embedding space and uses this latent state as the next input. It further shares KV-cache working memory across agents as a training-free communication channel. While effective, these training-free methods still rely on hand-crafted switching or local confidence heuristics and do not explicitly separate exploratory reasoning from rigorous derivation.

**Training-based latent reasoning.** Coconut [4] pioneers autoregressive latent reasoning by feeding the last-layer hidden state back as the next input embedding, showing that continuous thoughts can support implicit breadth-first exploration. However, its reliance on fixed latent steps and direct hidden-state reuse exposes a mismatch between hidden states and token embeddings. Subsequent methods improve this paradigm by learning better latent interfaces. SoftCoT [7] reduces full-model adaptation by using an assistant model to generate soft thoughts and a trainable projection module to align them with the target LLM. More recent methods further refine the definition and training of latent tokens. Latent-SFT [10] constrains latent reasoning to the vocabulary column space and learns latent tokens with KL and CE objectives, whereas CoLaR [11] predicts compressed embedding distributions and applies reinforcement learning to encourage both diverse exploration and compact reasoning. These methods demonstrate that latent reasoning can substantially shorten reasoning chains, but they largely aim to replace explicit CoT with latent computation. As a result, their performance can degrade on complex tasks where precise symbolic verification is essential.

## 5 Conclusion

We introduce LaTER, a latent-then-explicit reasoning paradigm for reducing test-time token cost without discarding explicit verification. The method separates reasoning into two phases: a continuous latent rollout for early exploration, followed by discrete CoT generation for symbolic checking and final-answer construction. In the training-free setting, we have found that pretrained reasoning models already exhibit structured latent trajectories, including terminating-token probes and informative entropy dynamics. A simple adaptive switch based on these signals reduces token usage and can improve accuracy on several benchmarks. We then construct LATENT-SWITCH-69K and train a LaTER model with a learned latent projector and halting supervision. The trained model improves the accuracy–efficiency tradeoff across mathematics, coding, and knowledge-intensive benchmarks, reaching 80.0% on AIME 2025 while using one third fewer tokens than standard CoT.

LaTER also leaves open important questions. The training-free switch is still a hand-crafted approximation, and the trained model’s behavior depends on the latent-budget distribution and the quality of distilled intuitions. Future work should learn richer instance-adaptive halting policies, study longer latent exploration for open-ended tasks, and extend latent-then-explicit reasoning to multimodal settings where full verbalization can be especially costly.

## References

- [1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022. URL [https://proceedings.neurips.cc/paper\\_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf).
- [2] Daya Guo, Dejian Yang, Haowei Zhang, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, September 2025. doi: 10.1038/s41586-025-09422-z. URL <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. URL [https://proceedings.neurips.cc/paper\\_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf).
- [4] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason E Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=Itxz7S4Ip3>.
- [5] Zhen Zhang, Xuehai He, Weixiang Yan, Ao Shen, Chenyang Zhao, and Xin Eric Wang. Soft thinking: Unlocking the reasoning potential of LLMs in continuous concept space. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=ByQdHPGKgU>.
- [6] Jiaru Zou, Xiyuan Yang, Ruizhong Qiu, Gaotang Li, Katherine Tieu, Pan Lu, Ke Shen, Hanghang Tong, Yejin Choi, Jingrui He, James Zou, Mengdi Wang, and Ling Yang. Latent collaboration in multi-agent systems, 2025. URL <https://arxiv.org/abs/2511.20639>.
- [7] Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. SoftCoT: Soft chain-of-thought for efficient reasoning with LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vienna, Austria, July 2025. doi: 10.18653/v1/2025.acl-long.1137. URL <https://aclanthology.org/2025.acl-long.1137/>.
- [8] Xinlei Yu, Zhangquan Chen, Yongbo He, Tianyu Fu, Cheng Yang, Chengming Xu, Yue Ma, Xiaobin Hu, Zhe Cao, Jie Xu, Guibin Zhang, Jiale Tao, Jiayi Zhang, Siyuan Ma, Kaituo Feng, Haojie Huang, Youxing Li, Ronghao Chen, Huacan Wang, Chenglin Wu, Zikun Su, Xiaogang Xu, Kelu Yao, Kun Wang, Chen Gao, Yue Liao, Ruqi Huang, Tao Jin, Cheng Tan, Jiangning Zhang, Wenqi Ren, Yanwei Fu, Yong Liu, Yu Wang, Xiangyu Yue, Yu-Gang Jiang, and Shuicheng Yan. The latent space: Foundation, evolution, mechanism, ability, and outlook, 2026. URL <https://arxiv.org/abs/2604.02029>.
- [9] Rui-Jie Zhu, Tianhao Peng, Tianhao Cheng, Xingwei Qu, Jinfa Huang, Dawei Zhu, Hao Wang, Kaiwen Xue, Xuanliang Zhang, Yong Shan, Tianle Cai, Taylor Kergan, Assel Kembay, Andrew Smith, Chenghua Lin, Binh Nguyen, Yuqi Pan, Yuhong Chou, Zefan Cai, Zhenhe Wu, Yongchi Zhao, Tianyu Liu, Jian Yang, Wangchunshu Zhou, Chujie Zheng, Chongxuan Li, Yuyin Zhou, Zhoujun Li, Zhaoxiang Zhang, Jiaheng Liu, Ge Zhang, Wenhao Huang, and Jason Eshraghian. A survey on latent reasoning, 2025. URL <https://arxiv.org/abs/2507.06203>.
- [10] Jingcheng Deng, Liang Pang, Zihao Wei, Shicheng Xu, Zenghao Duan, Kun Xu, Yang Song, Huawei Shen, and Xueqi Cheng. Latent reasoning in llms as a vocabulary-space superposition, 2025. URL <https://arxiv.org/abs/2510.15522v1>.

- [11] Wenhui Tan, Jiaze Li, Jianzhong Ju, Zhenbo Luo, Ruihua Song, and Jian Luan. Think silently, think fast: Dynamic latent compression of LLM reasoning chains. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=AQsko3PPUe>.
- [12] Zhenyi Shen, Hanqi Yan, Linhai Zhang, Zhanghao Hu, Yali Du, and Yulan He. CODI: Compressing chain-of-thought into continuous space via self-distillation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Suzhou, China, November 2025. doi: 10.18653/v1/2025.emnlp-main.36. URL <https://aclanthology.org/2025.emnlp-main.36/>.
- [13] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, et al. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [14] Team Olmo, Allyson Ettinger, Amanda Bertsch, et al. Olmo 3, 2026. URL <https://arxiv.org/abs/2512.13961>.
- [15] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions, February 2025. URL <https://matharena.ai/>.
- [16] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- [17] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [18] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- [19] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*, 2018.
- [20] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=1qv610Cu7>.
- [21] Jiawei Liu, Songrun Xie, Junhao Wang, Yuxiang Wei, Yifeng Ding, and Lingming Zhang. Evaluating language models for efficient code generation. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=IBCBMeAhmC>.
- [22] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models, 2026. URL <https://arxiv.org/abs/2601.18734>.
- [23] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- [24] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023. URL <https://arxiv.org/abs/2307.08691>.
- [25] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC ’20. IEEE Press, 2020.

- [26] Zhen Zhang, Xuehai He, Weixiang Yan, Ao Shen, Chenyang Zhao, and Xin Eric Wang. Soft thinking: Unlocking the reasoning potential of LLMs in continuous concept space. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=ByQdHPGKgU>.
- [27] Dachuan Shi, Abedelkadir Asi, Keying Li, Xiangchi Yuan, Leyan Pan, Wenke Lee, and Wen Xiao. Swireasoning: Switch-thinking in latent and explicit for pareto-superior reasoning llms, 2026. URL <https://arxiv.org/abs/2510.05069>.
- [28] Renyu Fu and Guibo Luo. Selar: Selective latent reasoning in large language models, 2026. URL <https://arxiv.org/abs/2604.08299>.

## A Per-step entropy heterogeneity in training-free latent reasoning

We provide a finer-grained view of the stopping signals used by the training-free version of LaTER. For each AIME 2025 problem, we begin with latent reasoning and continue rolling the hidden state forward until the current hidden state can be decoded to a model-native terminating token, such as `<|im_end|>`. At every latent step, we decode the hidden state into the vocabulary space and record the entropy of the resulting predictive distribution. This produces a trajectory-level entropy trace for the entire latent phase, from the first latent transition to the step immediately preceding termination.

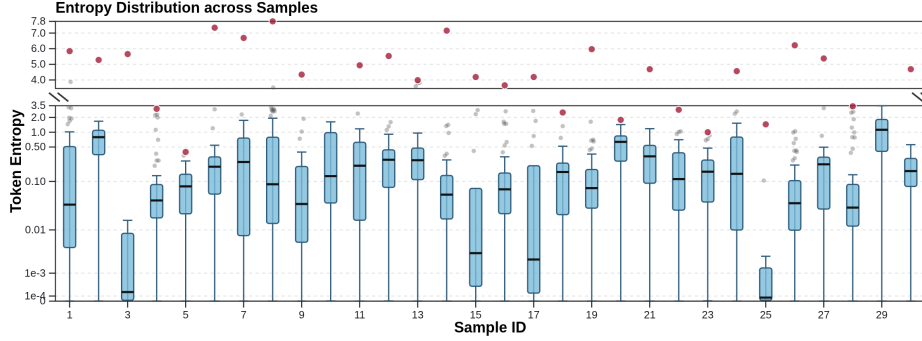


Figure 8: Per-step entropy distributions during the training-free latent phase on AIME 2025. For each problem, we record the entropy of the decoded vocabulary distribution at every latent step until the hidden state decodes to a terminating token such as `<|im_end|>`. Each boxplot aggregates all problems that are still in the latent phase at that step position. The wide variation in spread and upper tails shows that both the scale and the timing of peak entropy differ substantially across instances, which helps explain why a single entropy threshold cannot be uniformly optimal.

Figure 8 groups the entropy traces by latent step position. Each boxplot shows the entropy values at one step, using all problems that are still in the latent phase at that point. The distributions vary a lot across steps and across problems. At many step positions, the spread is wide, the upper tail changes noticeably, and the number of active trajectories also changes as some problems terminate earlier than others. This means that the latent phase does not follow one shared entropy curve.

The main finding is that both the size of the entropy peak and the step at which it appears differ greatly across problems. Some problems show a sharp early peak and then settle quickly. Others stay at relatively low entropy for many steps and peak only near the stopping point. Still others remain broad and unstable until very late in the trajectory, suggesting that the model continues exploring for much longer. In short, the entropy maximum is highly instance-specific in both value and timing.

This is why a single global threshold is only a rough stopping rule. A threshold that works well for trajectories with large early spikes may stop too early on problems that need a longer latent phase. A higher threshold may fit those slower cases better, but then it may wait too long on problems that are already ready to switch. In practice, the decision should depend not only on the entropy at one step, but also on the trend of the trajectory: whether entropy is rising or falling, how long uncertainty lasts, and whether the decoded state is already close to a terminating token.

For this reason, we view the training-free entropy rule as a useful diagnostic rather than a complete solution. It already captures meaningful structure in the latent dynamics and yields strong efficiency gains in the main experiments. However, the appendix results also show that a hand-crafted threshold cannot fully match the diversity of real latent trajectories. A more natural next step is to learn an instance-adaptive switching policy that uses the full trajectory, instead of relying on one fixed scalar cutoff.

## B Construction of Latent-Switch-69K

This section describes how we build the supervised corpus used to train LaTER. Each retained example contains four parts: a user problem, a distilled solution intuition, a shortened explicit CoT,

and a final answer. The preprocessing pipeline turns this record into a latent-supervised fine-tuning example. In this format, the model first passes through a bounded latent segment and then returns to ordinary explicit reasoning. The same pipeline also builds a teacher-reference conversation. This allows us to combine token-level language modeling targets with teacher-distribution supervision on explicit reasoning and answer tokens.

Figure 9 summarizes the composition of the final corpus. Consistent with Table 2, the final training split contains 69,745 examples. Most examples are in the medium-difficulty bucket, which accounts for 65.5% of the data. Hard examples account for 25.0%, and easy examples account for 9.5%. At the domain level, the source mixture is dominated by mathematical and coding data: math contributes about 37% of examples and code about 34%, while science-oriented questions account for roughly 5%. The remaining examples mainly come from instruction-following and general knowledge-oriented prompts, so the retained corpus stays centered on reasoning-intensive tasks while preserving some diversity in format and topic. This imbalance is intentional. Medium-difficulty problems provide the cleanest signal for learning when to move from latent exploration to explicit verification. They are hard enough to require real reasoning, but usually not so noisy that distillation becomes unstable. The hard subset broadens coverage and exposes the model to longer reasoning chains. The easy subset helps stabilize training and preserves short-form answer behavior.

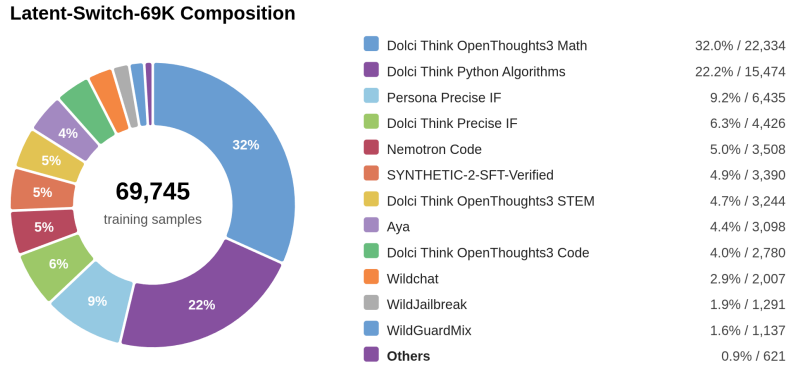


Figure 9: Composition of the Latent-Switch-69K training corpus after filtering and distillation.

**Distillation pipeline.** We start from reasoning traces sampled from Dolci-Think-SFT-32B. We then distill them with a stronger reasoning model. For each source problem, we first ask for a short *solution intuition* that states the high-level plan in a few sentences. We then ask the teacher to produce a shorter explicit CoT conditioned on the original problem and this intuition. The final record therefore keeps both a compressed latent-style summary and an explicit derivation. This design lets us train the model to use continuous latent computation without giving up token-level supervision on the visible reasoning segment.

**Student response template.** For a sample with  $m$  latent steps, distilled CoT tokens  $t_{1:n}$ , and answer tokens  $a_{1:r}$ , the assistant response is written as

`<latent_think>  $l_1, \dots, l_m$  </latent_think> <think>  $t_1, \dots, t_n$  </think>  $a_1, \dots, a_r$  <|im_end|>.`

The symbols  $l_1, \dots, l_m$  denote latent placeholder positions. In the current implementation, these positions are filled with a repeated placeholder token. However, they are not trained as ordinary language targets. During the forward pass, their token embeddings are replaced by recurrent latent states produced by the latent projector. The model therefore learns to reason internally across these positions instead of emitting visible text there.

**Latent budget assignment.** Each sample stores its latent budget as an integer field, which we denote conceptually as `n_latent_steps`. This value determines how many placeholder positions appear between `<latent_think>` and `</latent_think>`. In preprocessing, the budget is derived from the length of the distilled solution intuition. If the retained intuition contains  $L$  tokens, then the



target latent budget is set to about  $L/2$ . This value is then clipped by the maximum latent length and other tokenization constraints. This heuristic ties the amount of latent computation to the amount of compressed reasoning content kept by distillation. Across the final corpus, the latent-step count has mean 41.49 and median 40.00. This choice is determined by experimental phenomena in the training-free setting. In section 2.5, we found that the model naturally has the best reasoning accuracy and token efficiency after executing about 40-50 steps of latent reasoning, so we normalized the number of latent steps in the training data to this range.

**Supervision masks.** Let  $\mathcal{S}_{\text{prompt}}$  denote prompt positions. Let  $\mathcal{S}_{\text{lat}}^{\text{int}}$  denote latent interior positions, excluding the latent boundary markers. The CE label at token position  $i$  is

$$y_i = \begin{cases} -100, & i \in \mathcal{S}_{\text{prompt}} \cup \mathcal{S}_{\text{lat}}^{\text{int}}, \\ x_i, & \text{otherwise.} \end{cases}$$

Prompt tokens and latent interior placeholders are therefore masked from ordinary token-level CE. By contrast, the latent boundary tokens, explicit CoT tokens, answer tokens, and the terminal  $\langle \text{im\_end} \rangle$  token remain supervised. The preprocessing stage also builds dedicated masks for latent boundaries, explicit CoT regions, answer regions, and teacher-KL positions. This ensures that each objective is applied only where it is semantically meaningful.

**Teacher reference construction.** For teacher-distribution supervision, each example also includes a teacher-reference conversation. This reference omits the student’s latent placeholder segment and begins directly with the explicit reasoning part,

$$\langle \text{think} \rangle t_1, \dots, t_n \langle \text{/think} \rangle a_1, \dots, a_r.$$

Operationally, the teacher input pairs the original question with the distilled solution intuition. The shortened explicit CoT and final answer are then treated as the continuation to be matched. This provides teacher hidden states and teacher logits that align with explicit reasoning and answer positions. It does not require the teacher to model the student’s continuous latent placeholders. Teacher KL is applied only on positions selected by the teacher-KL mask.

Finally, the filtered corpus preserves the compression effect that motivates LaTER. As reported in Table 2, the distilled CoT compression ratio has mean 0.612 and median 0.569. The visible reasoning trace is therefore typically only about 57–61% as long as the original one. The dataset does not merely shorten responses. It explicitly separates condensed latent planning from explicit symbolic verification. That structural decomposition is what makes the later latent-reasoning finetuning objective well posed.

## C Training Details

We initialize LaTER from a Qwen3-14B backbone and optimize the model end to end so that it can interleave a continuous latent reasoning segment with an explicit textual reasoning segment. Each assistant response is formatted as

$$\langle \text{latent\_think} \rangle l_1, \dots, l_m \langle \text{/latent\_think} \rangle \langle \text{think} \rangle t_1, \dots, t_n \langle \text{/think} \rangle a \langle \text{im\_end} \rangle.$$

Here  $l_{1:m}$  denote latent placeholder positions,  $t_{1:n}$  denote explicit CoT tokens, and  $a$  denotes the final answer. During training, the latent placeholders are not treated as ordinary language targets. Instead, their token embeddings are replaced by recurrent latent states produced by a learned latent projector.

**Latent forward pass.** For each example, the model first processes the prompt and latent prefix with a cache-based recurrent rollout. At latent step  $t$ , the previous hidden state  $h_{t-1}$  is mapped by the latent projector to the next latent embedding,

$$e_t^{\text{lat}} = g_\phi(h_{t-1}).$$

These projected embeddings are then written back into the full input embedding sequence. The model subsequently performs a causal teacher-forcing forward pass over the entire sequence, using ordinary token embeddings outside the latent interior and projected latent embeddings inside the latent segment. During batch construction, we align both the assistant prefix and the transition into  $\langle \text{think} \rangle$  across examples, which keeps the latent-to-text boundary synchronized during distributed training.

**Supervised objectives.** Prompt tokens and latent interior placeholders are masked from token-level cross-entropy. The latent boundary tokens, explicit reasoning tokens, answer tokens, and the final `<|im_end|>` token remain supervised. We decompose the CE objective into an explicit-CoT term and a complementary non-CoT term,

$$\mathcal{L}_{\text{CE}} = \mathcal{L}_{\text{nonCoT}} + \lambda_{\text{CoT}} \mathcal{L}_{\text{CoT}}.$$

In the current configuration,  $\lambda_{\text{CoT}} = 0.5$ , so explicit reasoning tokens and the remaining supervised tokens contribute at the similar scale. The structural tags `<think>` and `</think>` are assigned to  $\mathcal{L}_{\text{nonCoT}}$ , whereas  $\mathcal{L}_{\text{CoT}}$  contains only the interior reasoning tokens.

**Teacher distribution matching.** In addition to CE, we apply cached teacher-distribution supervision on selected explicit reasoning and answer positions. Let  $q_i$  denote the cached top- $k$  teacher distribution and let  $p_\theta(\cdot | i)$  denote the student distribution at the aligned source position. The KL objective is

$$\mathcal{L}_{\text{KL}} = \frac{1}{|\mathcal{S}_{\text{KL}}|} \sum_{i \in \mathcal{S}_{\text{KL}}} D_{\text{KL}}(q_i \| p_\theta(\cdot | i)).$$

The current configuration uses temperature 1.0 and KL weight 0.25. This objective distills the teacher’s explicit reasoning and answer behavior without requiring the teacher to model the student’s continuous latent placeholders.

Importantly, we do not apply CE or KL supervision directly inside the latent reasoning segment. The latent placeholder positions are not trained to match token targets or teacher distributions. Instead, the latent segment is optimized only indirectly: gradients from the downstream explicit CoT and answer tokens are back-propagated through the latent rollout. In this way, the model learns latent reasoning states only to the extent that they help the later explicit reasoning and final answer.

**Halting supervision.** We further train the latent segment to terminate at the correct boundary with a dense auxiliary halting loss over latent interior positions. This loss compares the logit of `</latent_think>` and other forbidden structural tokens against the best allowed non-structural token, while also applying a BCE loss that toward the correct latent boundary. The halting weight is annealed to a small final value of 0.025 and gated by the CE quality signal so that the stopping objective does not dominate the language-modeling objective.

**Optimization setup.** The current training run uses bf16 training, FlashAttention-2, gradient checkpointing, DeepSpeed ZeRO-3, micro-batch size 1, and gradient accumulation 4. We set the maximum sequence length to 24,096, allow latent budgets of up to 128 steps.

**Compute resources.** All training runs are conducted on Nvidia A800 80GB GPUs. The trained LaTER model is optimized on a single node with 8 GPUs and requires approximately 5 days for the main training run under the configuration above. The CoT-SFT baseline is trained on the same hardware setup and requires approximately 2 days. All evaluation tasks are run on 2 A800 80GB GPUs. These numbers are intended to give a practical estimate of the wall-clock compute required to reproduce the reported training and evaluation pipeline.

**Evaluation protocol.** Unless otherwise specified, each reported evaluation result is obtained by running the corresponding model-setting pair multiple times under the same decoding configuration with a fixed random seed.

## D Prompts

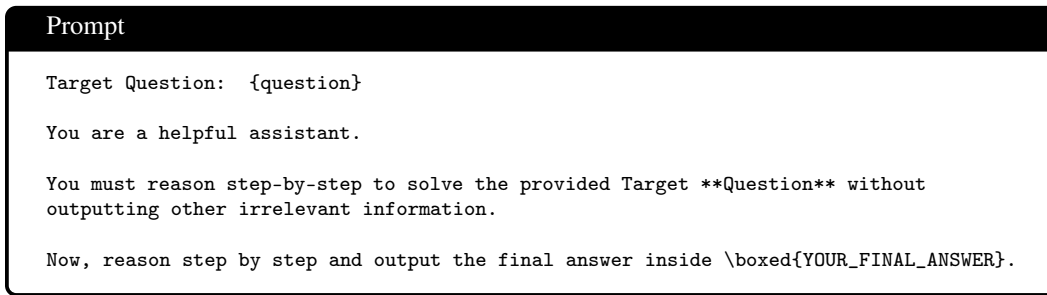


Figure 10: Prompt for math questions.

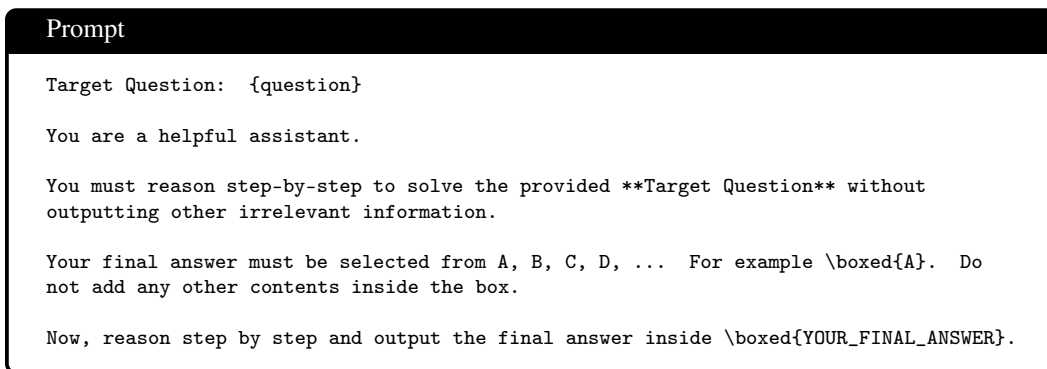


Figure 11: Prompt for multi-choice questions.

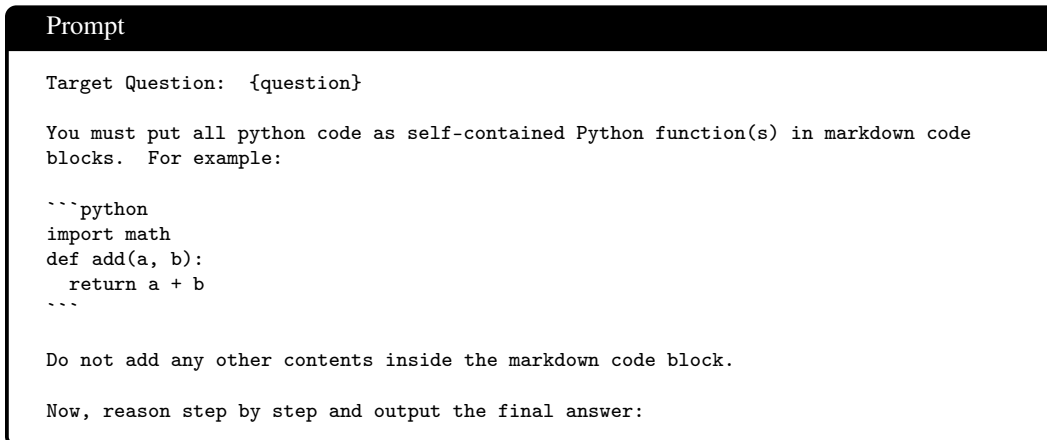


Figure 12: Prompt for code problems.

**Prompt**

You are an expert reasoning data curator.

Extract only the key insights from the source reasoning.

Rules:

- Return valid JSON only.
- Do not produce a short chain of thought.
- Do not provide any final answers in your response.
- correct\_insight must be the coarse but correct high-level solution idea.
- incorrect\_insights must only include wrong ideas that are explicitly evidenced in the source outputs.
- Do not invent errors that are not present in the source outputs.

Figure 13: System prompt to generate solution intuition.

**Prompt**

You will receive a problem, one or more source outputs, and optional ground truth.

Return JSON with this schema:

```
{
  "task_summary": "short task type description",
  "correct_insight": "2-10 sentences of high-level correct plan",
  "incorrect_insights": [
    {
      "idea": "2-10 sentences of a wrong high-level plan evidenced in the source outputs",
      "why_wrong": "brief reason it fails"
    }
  ],
  "source_answer_correct": true,
  "contains_reflection": true
}
```

REMEMBER: Only extract insights that reflect on the high-level idea. **\*\*Do not\*\*** give any actual answer in correct\_insight or incorrect\_insights.

Only give your complete plan to solve the question. Do not directly state whether it is right or wrong. Do not add content unrelated to the idea in correct\_insight or incorrect\_insights.

Problem prompt:

```
<<<PROMPT>>>
{prompt}
<<<END_PROMPT>>>
```

Source outputs:

```
<<<OUTPUTS>>>
{outputs}
<<<END_OUTPUTS>>>
```

Ground truth:

```
<<<GROUND_TRUTH>>>
{ground_truth}
<<<END_GROUND_TRUTH>>>
```

Metadata:

```
{metadata_json}
```

Figure 14: User prompt to generate solution intuition.

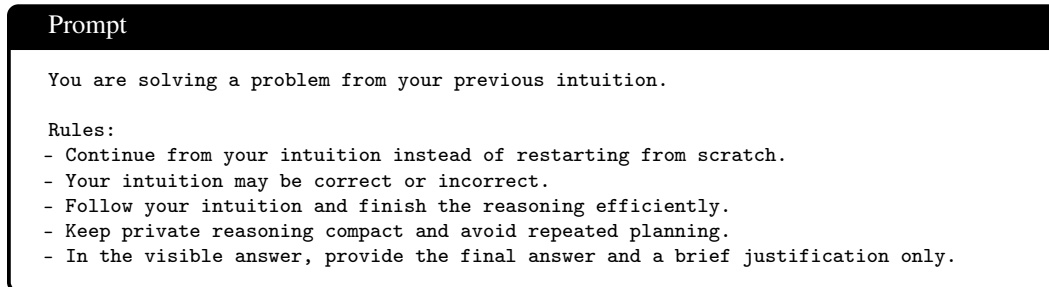


Figure 15: System prompt to generate shorter CoT.

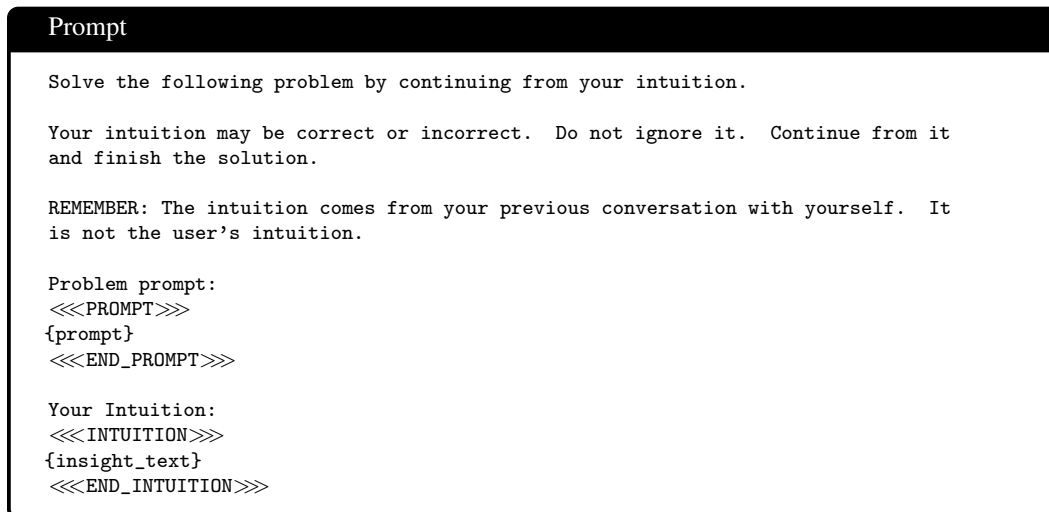


Figure 16: User prompt to generate shorter CoT.

## E Failure Case: Confident but Misleading Latent Reasoning

Figure 17 presents a representative failure case from AIME 2025 Problem 10. The problem asks the model to count valid fillings of a  $3 \times 9$  Sudoku-band grid, where each row and each  $3 \times 3$  block contains the digits  $1, \dots, 9$  exactly once. The correct answer is 81.

This example is notable because the latent trajectory does not appear uncertain under the entropy diagnostic. During latent reasoning, entropy remains low, with maximum entropy only 1.625. The highest entropy value in the CoT stage is 3.23. The latent reasoning ends after the trajectory hits `<|endoftext|>`. The subsequent explicit reasoning phase follows an incorrect counting path and returns 66.

In contrast, the same backbone under standard explicit CoT decoding solves the problem correctly and returns 81. The explicit CoT trace preserves the intermediate combinatorial bookkeeping: it fixes the first row, counts the possible second-row block assignments, accounts for the independent within-block permutations in the third row, and obtains  $9! \cdot 56 \cdot 6^6 = 2^{16} 3^{10} 5^1 7^2$ . The LaTER trace enters the explicit phase after a low-entropy latent segment, reconstructs a similar high-level counting setup, but then treats the final block as determined once the first two blocks are chosen. This omits an additional  $6^3$  ordering factor and yields  $9! \cdot 56 \cdot 6^3 = 2^{13} 3^7 5^1 7^2$ , producing the wrong answer 66.

This case illustrates a limitation of entropy as a standalone confidence signal for latent reasoning. Low entropy can indicate that the model is locally confident, but it does not guarantee that the latent state encodes the correct global combinatorial structure. Here, latent reasoning appears to compress the search into a confident but flawed state, while explicit CoT preserves enough intermediate structure to recover the missing factor.

Failure Case: AIME 2025 Problem 10

**Task.** The 27 cells of a  $3 \times 9$  grid are filled in using the numbers 1 through 9 so that each row contains 9 different numbers, and each of the three  $3 \times 3$  blocks heavily outlined in the example below contains 9 different numbers, as in the first three rows of a Sudoku puzzle.

```
1412181916131117151
1317191512111618141
1516111814171912131
```

The number of different ways to fill such a grid can be written as  $p^a \cdot q^b \cdot r^c \cdot s^d$ , where  $p, q, r$ , and  $s$  are distinct prime numbers and  $a, b, c$ , and  $d$  are positive integers. Find  $p \cdot a + q \cdot b + r \cdot c + s \cdot d$ .

**Baseline CoT.** The standard explicit CoT run explicitly enumerates the combinatorial factors. It first observes that the first row can be arbitrary ( $9!$  choices). Given the first row, it derives 56 possible set-level assignments for the second row and  $6^3$  within-block orderings. It then notes that the third row still has independent within-block orderings, contributing another  $6^3$  factor:

$$N_{\text{CoT}} = 9! \cdot 56 \cdot 6^6 = 2^{16} 3^{10} 5^1 7^2.$$

Therefore,

$$2 \cdot 16 + 3 \cdot 10 + 5 \cdot 1 + 7 \cdot 2 = 81.$$

The final prediction is correct:

$$\text{Pred} = 81, \quad \text{Gold} = 81.$$

The important qualitative point is that the explicit trace keeps the row/block assignment and ordering factors separate until the final factorization.

**Latent reasoning.** The output is organized as a latent segment followed by an explicit CoT segment. Abridged from the raw trace:

$$\begin{array}{c} \cdots \\ \text{low-entropy latent tokens} \\ H_{\max} = 1.625 \end{array} \quad <|\text{endof text}|> <\text{think}>$$

Given Block A, the number of valid Block B's is  $56 \times 216 \dots$  Block C is uniquely determined ... Therefore, the total number of grids is  $9! \times 56 \times 216 \dots$  Hence, the final answer is 66.

Thus the explicit CoT after the latent segment follows a superficially similar counting strategy, but it treats the third block as uniquely determined once the first two blocks are chosen. This makes the trace use

$$N_{\text{latent}} = 9! \cdot 56 \cdot 6^3 = 2^{13} 3^7 5^1 7^2$$

instead of  $9! \cdot 56 \cdot 6^6$ . This gives

$$2 \cdot 13 + 3 \cdot 7 + 5 \cdot 1 + 7 \cdot 2 = 66.$$

The final prediction is incorrect:

$$\text{Pred} = 66, \quad \text{Gold} = 81.$$

Figure 17: Low-entropy latent reasoning can still fail. On AIME 2025 Problem 10, the baseline explicit CoT run answers correctly, whereas the LaTER run remains low-entropy during latent reasoning but converges to an incorrect answer. This suggests that entropy is not sufficient to certify correctness of compressed latent reasoning states.

## F Licenses for Existing Assets

The table below summarizes the external models, datasets, and software explicitly used in this paper. For each asset, we list the original owner, how it is used, and the released license or additional usage condition stated by the official model card, dataset card, or repository. We use these assets for research and evaluation, cite their original sources in the reference, and do not redistribute third-party assets outside their original release terms.

Table 4: Existing external assets used in this paper, with their original owners and released license or usage terms.

| Asset                        | Type      | Use in this paper                                                                            | Original owner / credit                             | License       |
|------------------------------|-----------|----------------------------------------------------------------------------------------------|-----------------------------------------------------|---------------|
| Qwen3-14B                    | Model     | Main backbone for training-free experiments and the initialized base model for trained LaTER | Qwen Team; Yang et al. [13]                         | Apache-2.0.   |
| DeepSeek-R1-Distill-Llama-8B | Model     | Training-free comparison model                                                               | DeepSeek-AI; Guo et al. [2]                         | MIT.          |
| OLMo-3-32B-Think             | Model     | Training-free comparison model                                                               | AI2 / OLMo Team; Olmo et al. [14]                   | Apache-2.0.   |
| AIME 2025 via MathArena      | Benchmark | Main math evaluation benchmark                                                               | MathArena / ETH SRI Lab; Balunović et al. [15]      | MIT.          |
| MATH-500                     | Benchmark | Math evaluation benchmark                                                                    | OpenAI / HuggingFaceH4; Lightman et al. [16]        | MIT.          |
| Dolci-Think-SFT-32B          | Dataset   | Source dataset used to sample reasoning traces for constructing LATENT-SWITCH-69K            | AI2 / OLMo Team; Olmo et al. [14]                   | odc-by.       |
| GSM8K                        | Dataset   | Math word-problem benchmark                                                                  | OpenAI; Cobbe et al. [17]                           | MIT.          |
| GPQA                         | Dataset   | Science QA benchmark                                                                         | Rein et al. [18]                                    | MIT.          |
| ARC-Challenge                | Dataset   | Reasoning benchmark                                                                          | AI2; Clark et al. [19]                              | MIT.          |
| HumanEval+                   | Dataset   | Code-generation benchmark                                                                    | EvalPlus; Liu et al. [20]                           | Apache-2.0.   |
| MBPP+                        | Dataset   | Code-generation benchmark                                                                    | EvalPlus; Liu et al. [20, 21]                       | Apache-2.0.   |
| FlashAttention-2             | Software  | Attention kernel used in training implementation                                             | Dao-AILab; Dao [24]                                 | BSD-3-Clause. |
| DeepSpeed (ZeRO-3)           | Software  | Distributed training runtime                                                                 | Microsoft / DeepSpeed Team; Rajbhandari et al. [25] | Apache-2.0.   |